

Constructor Overloading – Employee Data

Experiment No: 04

Date: 19/09/2026

■ Objective / Aim

To design and implement an Employee Data program in Java demonstrating constructor overloading, where multiple constructors in the same class handle different sets of initialization data for an Employee object.

■ System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

■ Algorithm / Logic Description

1. Create a class named **Employee**.
2. Define attributes: empId, name, department, salary.
3. Define a **default constructor** that initializes attributes with default values.
4. Define a **parameterized constructor** taking empId and name only (for a newly onboarded employee with no department/salary assigned yet).
5. Define an **overloaded constructor** taking empId, name, and department (department assigned, salary pending).
6. Define a **fully parameterized constructor** taking empId, name, department, and salary.
7. Define a method **displayDetails()** to print employee information.
8. In main(), create multiple Employee objects using different constructors to demonstrate constructor overloading.
9. Display details of each employee object.

■ UML Class Diagram

```
+-----+
|           Employee           |
+-----+
| - empId: int                 |
| - name: String               |
| - department: String         |
| - salary: double             |
+-----+
| + Employee()                 |
| + Employee(empId, name)      |
| + Employee(empId, name, dept)|
| + Employee(empId, name, dept,|
|         salary)              |
| + displayDetails(): void     |
+-----+
```

Relationship: All four constructors belong to the same Employee class – this is constructor overloading (same class, same constructor name, different parameter lists).

■ Source Code

```
class Employee {
    private int empId;
    private String name;
    private String department;
    private double salary;

    // 1. Default constructor
    Employee() {
        this.empId = 0;
        this.name = "Not Assigned";
        this.department = "Not Assigned";
        this.salary = 0.0;
    }

    // 2. Constructor with empId and name only
    Employee(int empId, String name) {
        this.empId = empId;
        this.name = name;
        this.department = "Not Assigned";
        this.salary = 0.0;
    }

    // 3. Constructor with empId, name, and department
    Employee(int empId, String name, String department) {
        this.empId = empId;
        this.name = name;
        this.department = department;
        this.salary = 0.0;
    }

    // 4. Fully parameterized constructor
    Employee(int empId, String name, String department, double salary) {
        this.empId = empId;
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    // Display employee details
    public void displayDetails() {
        System.out.println("Employee ID : " + empId);
        System.out.println("Name      : " + name);
        System.out.println("Department : " + department);
        System.out.printf("Salary      : Rs.%.2f%n", salary);
    }
}

// Main class
public class EmployeeDataDemo {
    public static void main(String[] args) {
        System.out.println("===== EMPLOYEE 1 (Default Constructor) =====");
        Employee emp1 = new Employee();
        emp1.displayDetails();

        System.out.println("\n===== EMPLOYEE 2 (empId, name) =====");
        Employee emp2 = new Employee(101, "Ravi Kumar");
        emp2.displayDetails();

        System.out.println("\n===== EMPLOYEE 3 (empId, name, department) =====");
        Employee emp3 = new Employee(102, "Sneha Reddy", "Human Resources");
        emp3.displayDetails();

        System.out.println("\n===== EMPLOYEE 4 (Fully Parameterized) =====");
        Employee emp4 = new Employee(103, "Arjun Rao", "Engineering", 65000.00);
        emp4.displayDetails();
    }
}
```

■ Compilation & Execution Commands

Save the source code exactly as:

EmployeeDataDemo.java

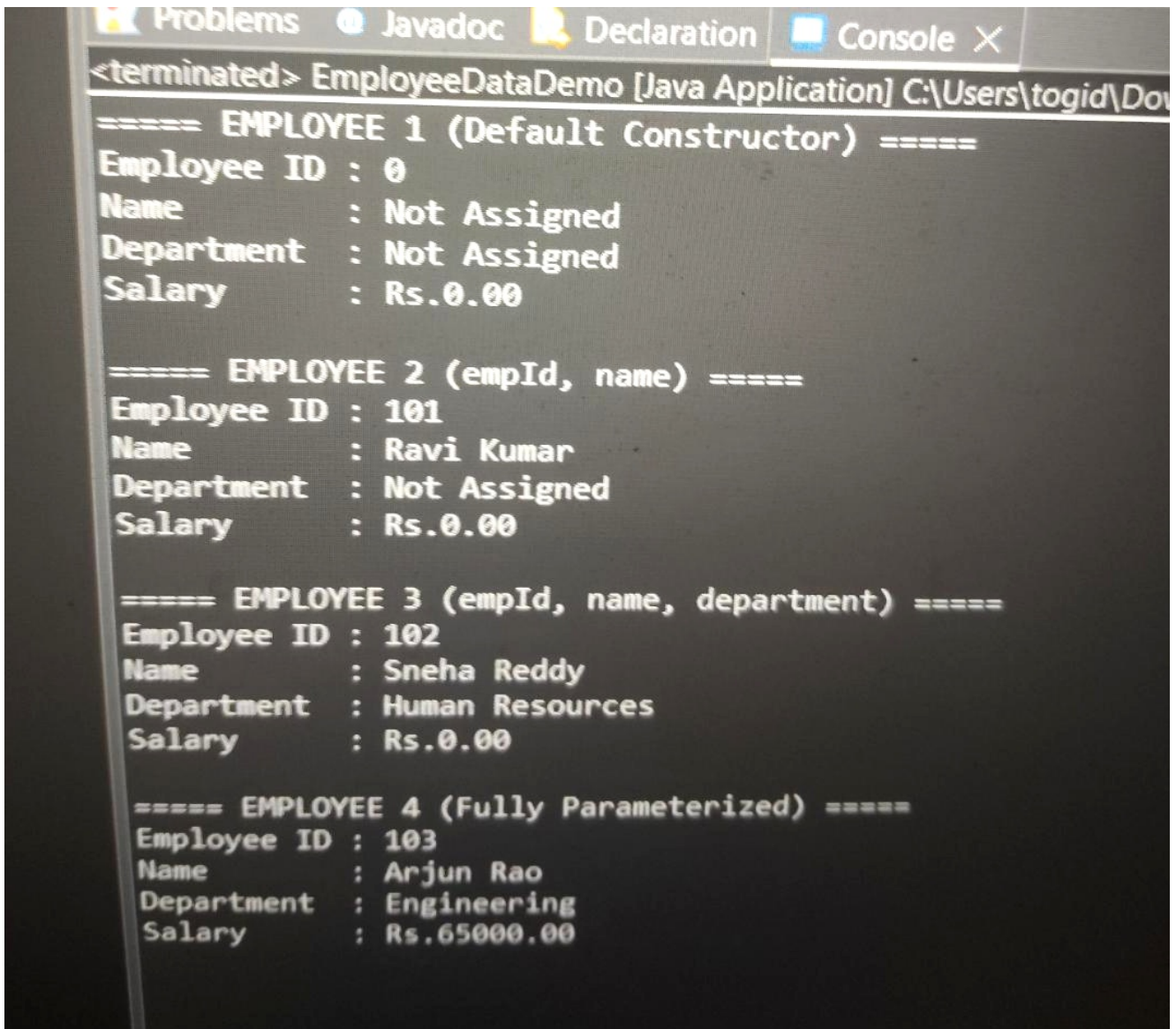
Compile the program:

```
javac EmployeeDataDemo.java
```

Run the compiled program:

```
java EmployeeDataDemo
```

■ Output Screen



```
<terminated> EmployeeDataDemo [Java Application] C:\Users\togid\Do
===== EMPLOYEE 1 (Default Constructor) =====
Employee ID : 0
Name       : Not Assigned
Department : Not Assigned
Salary    : Rs.0.00

===== EMPLOYEE 2 (empId, name) =====
Employee ID : 101
Name       : Ravi Kumar
Department : Not Assigned
Salary    : Rs.0.00

===== EMPLOYEE 3 (empId, name, department) =====
Employee ID : 102
Name       : Sneha Reddy
Department : Human Resources
Salary    : Rs.0.00

===== EMPLOYEE 4 (Fully Parameterized) =====
Employee ID : 103
Name       : Arjun Rao
Department : Engineering
Salary    : Rs.65000.00
```

■ Result

The Employee Data program was successfully implemented in Java, demonstrating constructor overloading through four different constructors of the Employee class, each handling a different combination of initialization parameters.